

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Mock Interview (Low)

Assessment Tool Weightage: 30%

Faculty Name: Dr.G.Ayyappan

Student Reg.No: 192571049

Name: Hemalakshmi H

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Interviewer: 'Explain the steps of query processing in DBMS. How does a query optimizer decide the best execution plan?'	BL3 – Apply	5
2	Interviewer: 'What is the difference between heuristic optimization and cost-based optimization? When would you use each?'	BL4 – Analyze	5
3	Interviewer: 'How would you implement database connectivity in a real project? Explain the difference between JDBC and ODBC.'	BL3 – Apply	5
4	Interviewer: 'What are ACID properties? Give a real-world example where each property is critical.'	BL3 – Apply	5
5	Interviewer: 'Explain concurrency control problems: dirty read, lost update, and unrepeatable read. How does 2PL solve them?'	BL4 – Analyze	5
6	Interviewer: 'What is deadlock in DBMS? Describe the deadlock detection algorithm and at least two prevention strategies.'	BL4 – Analyze	5
7	Interviewer: 'Compare Immediate Update and Deferred Update recovery. In what scenarios is each preferred?'	BL4 – Analyze	5

8	Interviewer: 'Explain Shadow Paging. What are its advantages and limitations compared to log-based recovery?'	BL3 – Apply	5
9	Interviewer: 'What is a serializability schedule? How do you test for conflict serializability using a precedence graph?'	BL4 – Analyze	5
10	Interviewer: 'Describe the Two-Phase Locking protocol variations: Basic 2PL, Strict 2PL, and Rigorous 2PL. Which is most commonly used and why?'	BL4 – Analyze	5

Code of Conduct:

I Hemalakshmi H - 192571049 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated

Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately

1. Query processing & how optimizer chooses a plan

Steps of query processing:

1. **Parsing:**
 - SQL is checked for syntax and converted into a parse tree.
2. **Query rewrite / relational algebra:**
 - The parse tree is transformed into relational algebra; simple rewrites (like removing redundant predicates) are applied.
3. **Logical optimization (heuristic):**
 - Apply rules like push selections down, project early, and reorder joins logically.
4. **Cost-based optimization:**
 - Generate alternative logical/physical plans and estimate their cost (I/O, CPU, memory).
5. **Plan selection:**
 - Choose the plan with the **lowest estimated cost**.
6. **Execution:**
 - The chosen plan is executed using physical operators (index scan, hash join, sort-merge join, etc.).
7. **Result formatting:**
 - Results are returned to the client in the requested format.

How optimizer decides best plan:

- It **enumerates candidate plans** (different join orders, different access paths).
- For each plan, it uses **statistics** (table size, index selectivity, histogram on columns) to estimate:
 - Number of tuples
 - Number of pages
 - I/O cost and CPU cost
- It picks the plan with **minimum estimated cost**—this is the “best” execution plan.

2. Heuristic vs cost-based optimization

Heuristic optimization:

- Uses **rule-based transformations**:
 - Push WHERE conditions close to base tables.
 - Project only needed columns early.
 - Join smaller relations first.
- Does **not** rely on detailed statistics or numeric cost.
- Fast, simple, and good as a **first pass**.

Cost-based optimization:

- Uses **statistics** (cardinality, selectivity, index info).
- Enumerates multiple plans and **computes estimated cost** for each.
- Chooses the plan with **lowest cost** (I/O, CPU).

When to use each:

- **Heuristic**:
 - As a quick, general improvement step; when stats are poor or missing.
- **Cost-based**:
 - For serious performance tuning; in mature systems with good statistics; for complex queries where join order and access path matter a lot.

3. Implementing database connectivity; JDBC vs ODBC

Implementing connectivity in a real project:

- Use a **connection pool** (e.g., HikariCP in Java, or built-in pool in frameworks).
- Steps:
 1. Configure DB URL, username, password.
 2. Get a connection from the pool.
 3. Use **parameterized queries** (PreparedStatement) to avoid SQL injection.
 4. Execute query, process ResultSet.
 5. Commit or rollback transaction.
 6. Return connection to pool.

JDBC vs ODBC:

- **JDBC (Java Database Connectivity):**
 - Java-specific API.
 - Uses **JDBC drivers** written in Java.
 - Designed for Java applications; integrates well with Java frameworks.
- **ODBC (Open Database Connectivity):**
 - Language-independent, C-based API.
 - Uses **ODBC drivers**; can be used from many languages via bindings.
 - More generic, often used in Windows environments and legacy systems.

Short line: JDBC = Java-centric, higher-level; ODBC = language-neutral, lower-level, older standard.

4. ACID properties with real-world examples

ACID:

1. **Atomicity:**
 - All steps of a transaction either **fully happen** or **not at all**.
 - Example: Money transfer—debit from A and credit to B must both succeed or both fail.
2. **Consistency:**
 - Transaction moves the database from one **valid state** to another, respecting constraints.
 - Example: After transfer, total money in the system remains the same; no negative balances if constraint forbids it.
3. **Isolation:**
 - Concurrent transactions behave as if they ran **one after another** (serially).
 - Example: While one transfer is in progress, another user reading balance doesn't see half-updated data.
4. **Durability:**
 - Once a transaction commits, its effects **survive crashes**.
 - Example: After transfer success message, even if power fails, the new balances are stored.

5. Concurrency problems & how 2PL solves them

Problems:

1. Dirty read:

- Transaction T2 reads data written by T1 that is **not yet committed**. If T1 rolls back, T2 has used invalid data.

2. Lost update:

- T1 and T2 both read same value, update it, and one update **overwrites** the other.

3. Unrepeatable read:

- T1 reads a row twice; between reads, T2 modifies it, so T1 sees **different values**.

How 2PL (Two-Phase Locking) solves them:

- Transactions **acquire locks** (shared for read, exclusive for write) in a **growing phase**, then **release** them in a **shrinking phase**.
- Dirty read:**
 - Reader must acquire shared lock; writer holds exclusive lock until commit → no one can read uncommitted changes.
- Lost update:**
 - Exclusive locks prevent two writers from updating the same item simultaneously.
- Unrepeatable read:**
 - Shared lock held for duration of transaction prevents other transactions from writing the same item until lock is released.

6. Deadlock, detection, and prevention strategies

Deadlock:

- Occurs when two or more transactions **wait on each other's locks** in a cycle—none can proceed.

Deadlock detection (Wait-For Graph):

- Build a **Wait-For Graph (WFG)**:

- Node = transaction.
- Edge $T1 \rightarrow T2$ if $T1$ is waiting for a resource held by $T2$.
- 2. Periodically run **cycle detection** (DFS or similar).
- 3. If a cycle exists → **deadlock detected**.
- 4. Choose a **victim transaction** (e.g., least work done, lowest priority), abort it, release its locks.

Prevention strategies:

1. **Wait-Die (timestamp-based):**
 - Older transaction may wait; younger transaction requesting a lock held by older **dies** (is rolled back).
2. **Wound-Wait:**
 - Older transaction “wounds” younger by forcing it to rollback; older never waits for younger.
3. **Timeout:**
 - If a transaction waits too long for a lock, it is aborted.
4. **Resource ordering:**
 - Enforce a global order on lock acquisition to avoid cycles.

7. Immediate Update vs Deferred Update recovery

Immediate Update:

- Changes can be written to the database **before commit**.
- Requires **UNDO and REDO**:
 - UNDO for uncommitted transactions.
 - REDO for committed transactions.
- More flexible; supports high concurrency and early writes.

Deferred Update:

- Changes are **not written** to the database until **after commit**.
- Needs only **REDO**:
 - No UNDO, because uncommitted changes never reach the database.
- Simpler recovery logic; good when you can buffer changes until commit.

Preferred scenarios:

- **Immediate Update:**
 - High concurrency systems, long transactions, where you want to flush changes early.
- **Deferred Update:**
 - Systems where commit is frequent and buffering is cheap; simpler recovery, fewer UNDO operations.

8. Shadow Paging: advantages & limitations

Shadow Paging:

- Maintain two page tables:
 - **Shadow page table:** stable, committed state.
 - **Current page table:** used for ongoing transaction updates.
- On commit:
 - Current page table becomes the new shadow.
- On failure:
 - Discard current page table and revert to shadow → instant rollback.

Advantages:

- No need for complex log-based UNDO/REDO.
- Recovery is fast: just switch back to shadow page table.
- Pages not touched by transaction remain unchanged.

Limitations:

- Managing and copying page tables can be **expensive** for large databases.
- Harder to support **fine-grained concurrency** and multiple transactions.
- Less flexible than log-based recovery (like ARIES), especially for complex workloads and partial rollbacks.

9. Serializability schedule & conflict serializability test

Serializability schedule:

- A schedule of transactions is **serializable** if its effect is equivalent to **some serial order** of those transactions.

Conflict serializability using precedence graph:

1. Identify **conflicting operations**:
 - Same data item, at least one is a write.
 - Types: R-W, W-R, W-W.
2. Build **precedence graph**:
 - Node = transaction.
 - Edge $T_i \rightarrow T_j$ if an operation of T_i conflicts with a **later** operation of T_j on the same item.
3. Check for **cycles**:
 - If graph has **no cycle** $\rightarrow$ schedule is **conflict-serializable**.
 - If graph has a cycle $\rightarrow$ **not conflict-serializable**.

10. Variants of 2PL: Basic, Strict, Rigorous

Basic 2PL:

- Two phases:
 - **Growing**: acquire locks.
 - **Shrinking**: release locks.
- Locks can be released **before commit**, as long as no new locks are acquired afterward.

Strict 2PL:

- All **write (exclusive) locks** are held **until commit or abort**.
- Read locks may be released earlier.
- Prevents **dirty reads** and **cascading aborts**.

Rigorous 2PL:

- Both **read and write locks** are held until **commit or abort**.

- Strongest form; ensures schedules are **strict and serializable**.

Most commonly used and why:

- **Strict 2PL** is most common:
 - Good balance: strong correctness (no dirty reads, no cascading aborts) with reasonable concurrency.
 - Rigorous 2PL is safer but more restrictive; basic 2PL allows more concurrency but can cause cascading aborts.